

HW2 - BPE Tokenizer for Named Entity Recognition

Raz Amzaleg - ID 206922478

Method

All three tokenizers use a word-level Byte Pair Encoding tokenizer (Sennrich/HF style, as in the lecture). Each line is split on whitespace and every non-initial word is prefixed with a space marker `_`; the vocabulary is initialized with all characters and then the most frequent adjacent symbol pair is merged repeatedly (computed with incremental pair-counts over a unique-word frequency

table, so full domain_1 trains in ~12 min) until the vocabulary reaches 2000. Encoding uses greedy longest-match (maximal-munch) over the trained vocabulary via a prefix trie - $O(\text{len})$ per word with a per-word cache; decoding joins token surfaces and maps `_` back to a space.

Differences from textbook BPE: (1) a bounded bigram phase adds the few most frequent adjacent word-pairs as single tokens (e.g. `in the`), which satisfies the mandatory " ≥ 1 bigram" rule while capping tokens at the bigram level (at most one internal space / two words); (2) length-preserving [UNK] decoding - an unseen character decodes to a single replacement character, so token $\leftrightarrow$ word character-span alignment in the NER pipeline never shifts; (3) case is preserved, since capitalization is the dominant entity cue on these data; (4) encoding is decoupled from merge-replay - greedy longest-match over the final vocab gives the same legal segmentation far faster (see Competition).

Tokenizers and the hidden domain

T1 is trained on domain_1 (Twitter), T2 on domain_2 (news). T3 targets a hidden domain and is trained

on domain_1 + domain_2 combined for maximum character and subword coverage. The vocabulary is kept

small (2000) deliberately: a data analysis showed ~68 % of dev entities are out-of-vocabulary, and small shared subwords generalize to unseen words far better than rare whole-word tokens - a whole token for a never-seen entity is an embedding the fixed NER model never trained. Length-preserving [UNK] further protects T3 against unknown characters in the hidden domain.

Most / least common bigrams (training set)

- domain_1 - most: `in the, _for the, _going to, _to be, _on the;`
least (freq 1): `_hockey_3-0, _ta_so, _Queen_will, _voice_as, @officialSusanB_ta.`
- domain_2 - most: `_of the, _in the, _to the, _on the, _to be;`
least (freq 1): `_a_Wönder, _10-year-old_self, _Kids:_Reflections, _Reflections_During, Giving_Culture.`

Validation (dev sets) and parameter selection

At the chosen vocab 2000: T1 (domain_1) F1 0.469, tok/char 0.339; T2 (domain_2) F1 0.961, tok/char 0.358. Both clear the 0.4 threshold; domain_2 has a large margin.

Vocabulary was chosen by a full sweep (num_bigrams = 5, full-data training, 20-epoch NER):

vocab	domain_1 F1	domain_2 F1	domain_1 tok/char	domain_2 tok/char
1500	0.473	0.960	0.361	0.390
2000	0.469	0.961	0.339	0.358
3000	0.450	0.961	0.314	0.321

This is an efficiency $\leftrightarrow$ F1 trade: larger vocab compresses better but lowers domain_1 F1 (smaller vocabularies force rare/entity words into *shared* subwords that generalize to the 68 %-OOV dev set).

We pick 2000 as the balanced operating point - best generalization proxy at competitive compression; 1500 is dominated, 3000 trades 4 % F1 for ~8 % compression. A num_bigrams sweep (5 $\rightarrow$ 200) gave essentially no compression gain and slightly more entity fragmentation, so we keep 5.

Competition: encoding speed

Switching encoding from merge-replay ($O(W^2)$ per word) to greedy longest-match over the final vocab

(prefix trie, $O(W)$) speeds up cold encoding by +54 % on domain_1 (363k $\rightarrow$ 560k tok/s) and +64 %

on domain_2 (430k $\rightarrow$ 706k tok/s) with slightly fewer tokens. The segmentation differs, costing a small F1 amount (T1 0.469 $\rightarrow$ 0.460, T2 0.961 $\rightarrow$ 0.960, both well above the 0.4 gate); given the equal-weight speed/efficiency/F1 competition, the large speed gain outweighs the ~2 % F1 dip, so longest-match is the shipped encoder.

Hidden-domain (T3) evaluation

T3 trains on all provided data (domain_1 + domain_2). We tested a balanced alternative (downsampling domain_1 to domain_2's size to de-bias the vocabulary); it lost on both dev proxies (domain_1 F1 0.488 $\rightarrow$ 0.469, domain_2 0.964 $\rightarrow$ 0.961) - discarding 75 % of domain_1 costs more in coverage

than the de-biasing gains, so the full-data mix is kept. Notably T3's domain_1 F1 (0.488) exceeds T1's

(0.469): more data yields better shared-subword coverage. For the unknown hidden domain we therefore

expect solid F1 - maximal subword coverage plus length-preserving [UNK] give every word a trainable,

alignment-safe first subtoken regardless of domain. The main limiter is the ambiguous capitalization cue (capitalized tokens begin both entities and non-entities), a property of the fixed model and the small, noisy data rather than of the tokenizer.